

REN-208: Applying Business Logic

Common Filters

Batch / De-batch Filter

The Batch / De-batch filter runs in two modes:

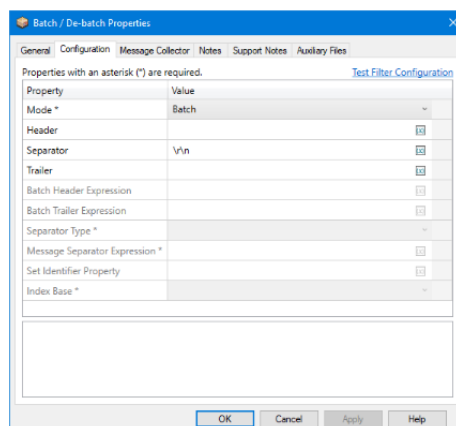
- **Batch** - incoming messages are combined into a single file
- **De-batch** - an incoming file is split into its component messages

Batch Mode

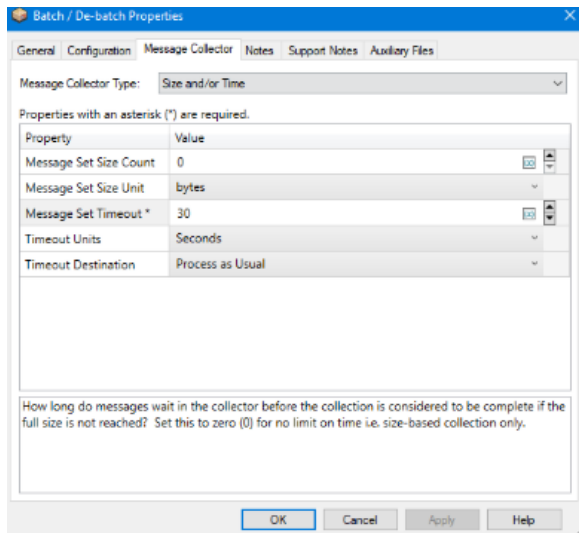
When set to Batch mode, the **Batch / De-batch** filter accepts incoming messages and combines them into a single file in accordance with the settings on the filter's **Message Collection** tab.

The configurable properties for the filter in Batch mode are:

- **Header** - the string that appears at the beginning of the batched file. This can include special formatting sequences \r (carriage return) and \n (line feed).
- **Separator** - the string that separates messages within the batched file. For example, if \r\n is specified, each message will be listed on a new line.
- **Trailer** - the string that appears at the end of the batched file.



Message Collector Settings



The way in which the output batched file is created is determined by the settings on the **Message Collector** tab of the **Batch/De-batch** filter. A quick way to test this functionality is to configure the filter as shown; this will create the output batched file at 30 second intervals.

Incoming text files will be batched into a single file using the following configuration, with the output file showing each record on a new line. The Message collector is covered in more detail in the *Putting it all together* module.

De-batch Mode

When set to De-batch mode, the Batch/De-batch filter accepts an incoming batched file and separates it into its component messages according to the filter's configured properties.

The configurable properties for the filter in De-batch mode are:

- **Batch Header Expression** - the string that appears at the beginning of the file to be de-batched.
- **Batch Trailer Expression** - the string that appears at the end of the file to be de-batched.
- **Separator Type** - options: Marks Start of Message, Separates Message. This property identifies whether messages are separated by a marker or separator
- Separators are the string between messages and do not appear in the de-batched message bodies.
- Markers are found at the beginning of the message and do appear in the de-batched content.
- **Message Separator Expression** - the POSIX (Portable Operating System Interface) regular expression that identifies the message separator. Note that if

the separator type is set to Marks Start of Message and you wish to de-batch on all instances of MSH|, you must escape the pipe | character using MSH\|.

An incoming batched HL7 message will be separated into its components by the following configuration:

Batch / De-batch Properties

General Configuration Message Collector Notes Support Notes Auxiliary Files

Properties with an asterisk (*) are required. [Test Filter Configuration](#)

Property	Value
Mode *	De-batch
Header	
Separator	
Trailer	
Batch Header Expression	BHS\
Batch Trailer Expression	BTS\
Separator Type *	Marks Start of Message
Message Separator Expression *	MSH\
Set Identifier Property	
Index Base *	Zero

OK Cancel Apply Help

The BHS (Batch Header Segment) and BTS (Batch Trailer Segment) segments are part of the HL7 standard and appear at the start and end of a batched HL7 message respectively.

XML Messages containing repeating elements can be de-batched using the filter. This is sometimes used with the Database Communication Point when operating in Input mode to de-batch records returned from a database. The Generating Messages From the Database lesson in the Databases module contains an example that illustrates this.

Where the incoming message is large, it is preferable to use the de-batch functionality of the Directory communication point if feasible. This is because the Batch / De-batch filter conducts its operations in memory and requires the complete message to be loaded into memory before the operation can occur. The Directory communication point conducts its de-batch operation on the input message as it is streamed to the route and utilizes memory more efficiently.

JavaScript Filter

The JavaScript filter executes the script specified in its configuration to process messages passing through the Rhapsody engine.

A full description of its functionality and configuration can be found in the *Messaging* module of this course.

Mapper Filter

The Mapper filter uses the Java Mapping Engine to map one or more input messages to an output message.

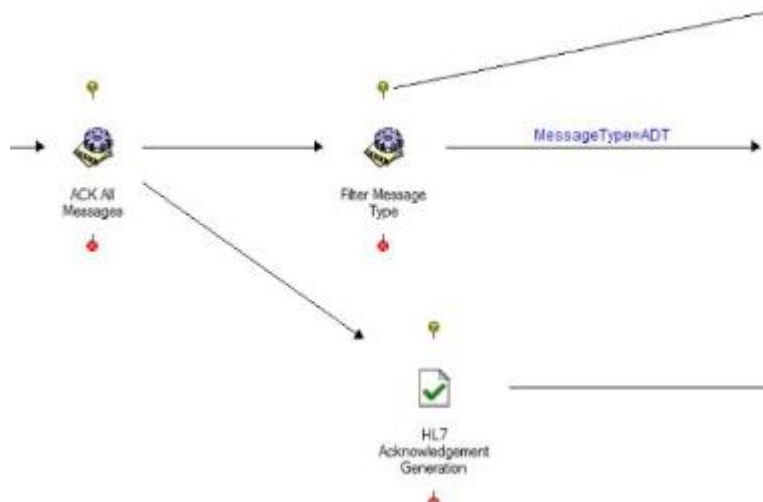
A full description of its functionality and configuration can be found in the *Messaging* module of this course.

No-operation Filter

The No-operation filter passes messages from input to output without change. It has no configurable properties and can be used in the following situations:

- Managing and clarifying the flow of messages through a route
- Creating route branches so that copies of an incoming message can be separately processed on different pathways
- Implementing a conditional branch in the message path, such that messages meeting the condition follow the normal output path and messages not matching the condition follow the **No Match**
- Applying **Message Collector** conditions (this option will be covered in Putting it all together module); in the case of the No-operation filter, this is typically used to delay messages or to ensure a sequence
- Providing "book-ends" around a route inter-connection to protect the message path

The use of No-operation filters will assist the quick, 'at-a-glance' evaluation of route layout, particularly when informative filter names have been provided to help others deduce the purpose of your route and understand its message flow.



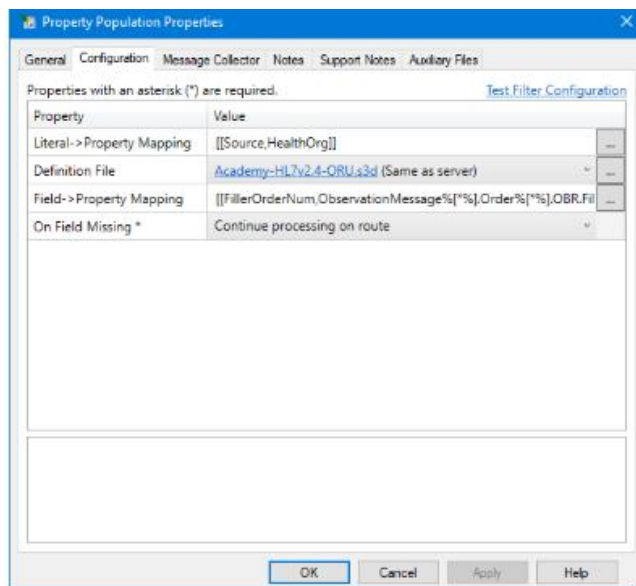
While there are many places where a No-operation filter will enhance your configuration, excessive use can reduce performance.

Property Population Filter

The Property Population filter is used to insert information into message properties from fields in the message or from explicit values entered during configuration.

Configuration Properties

- **Literal->Property Mapping** - used to identify the message properties that will be set to explicit values.
- **Definition File** - the message definition used in conjunction with the **Field->Property Mapping** property to identify those fields whose values are to be extracted.
- **Field->Property Mapping** - the field(s) in the message whose value(s) will be extracted for insertion into a message property.
- **On Field Missing** - the action to be taken should a specified field be missing from the incoming message. The options are to send the message to the filter's Error Connector or to ignore the missing field and continue processing the message.

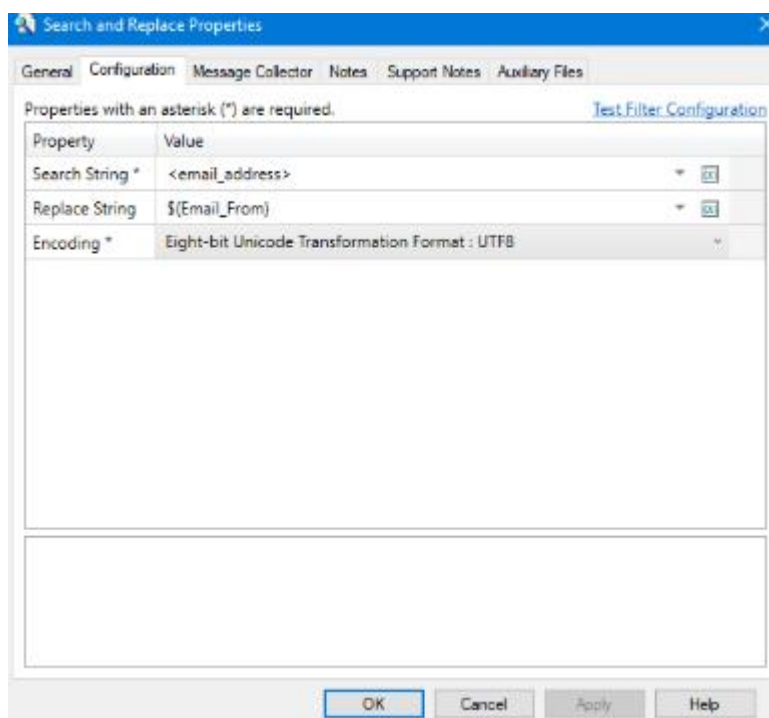


Search and Replace Filter

The Search and Replace filter performs simple text substitution on incoming messages.

- If a **Replace String** is specified, this filter locates all instances of the Search String and replaces it with the Replace String.
- If a **Replace String** is not specified, this filter locates and removes all instances of the specified Search String.

Configuration Properties



- **Search String** - the text to search for within the message.

- **Replace String** - the text which replaces the search string.
- **Encoding** - the character encoding of the search and replace strings. By default, this will be ASCII (American Standard Code for Information Interchange).

- **Worked Examples**

- A Rhapsody RLC containing worked examples for this set of filters is available for download [here](#):
- Each example is built on a separate route and the relevant details are noted in the Route Notes.
- The input for each route is proved by a Timer communication point to simplify input.
- All components are configured for manual start and will require starting to test the configuration.

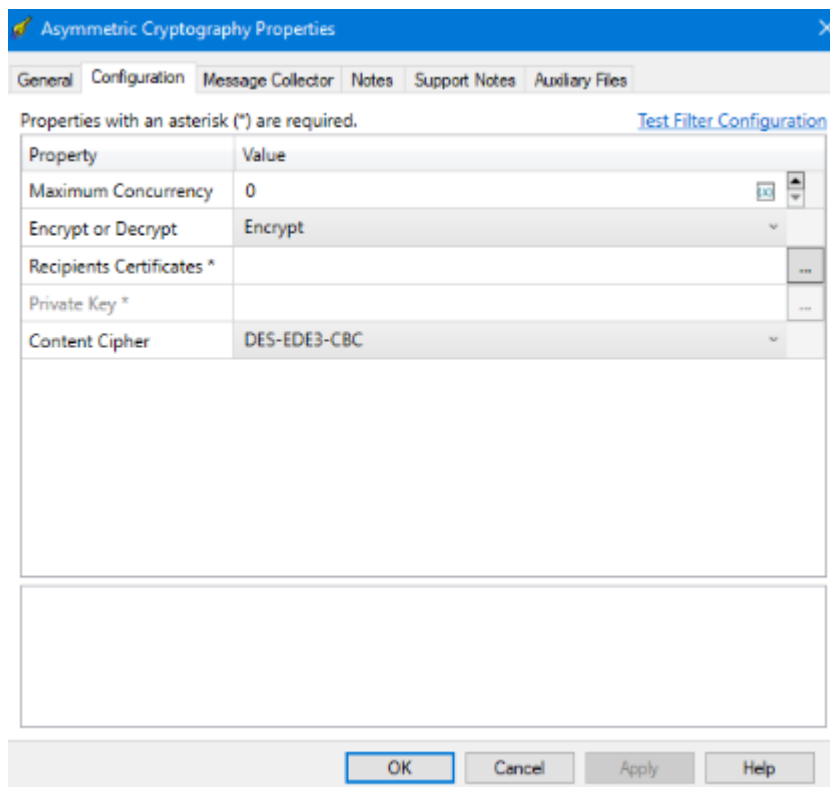
Utility Filters

Asymmetric Cryptography Filter

The Asymmetric Cryptography filter filter encrypts or decrypts messages using an asymmetric (public / private key) cryptography algorithm. The encryption operation generates a PKCS #7 Enveloped Message with a public key contained in the Certificate.

The filter uses certificates managed via the **Rhapsody Certificate Manager**, as discussed in Rhapsody Services.

Configuration



- **Maximum Concurrency** - the maximum number of instances of this filter that can be created and used in parallel. This property can be left at 0 (zero, which means an unlimited number of instances) for this filter, but overall system performance may be improved by setting a limit for the filter.
- **Encrypt or Decrypt** - sets whether the action of this filter is to encrypt or decrypt incoming messages.
- **Recipient's Certificates** - (only available in Encrypt mode). The certificate(s) containing the recipient's public key which is used to encrypt the message.
- **Private Key** - (only available in Decrypt mode). The recipient's private key used to decrypt the message.
- **Content Cipher** - (only available in Encrypt mode). The name of the encryption cipher used to encrypt the original message.

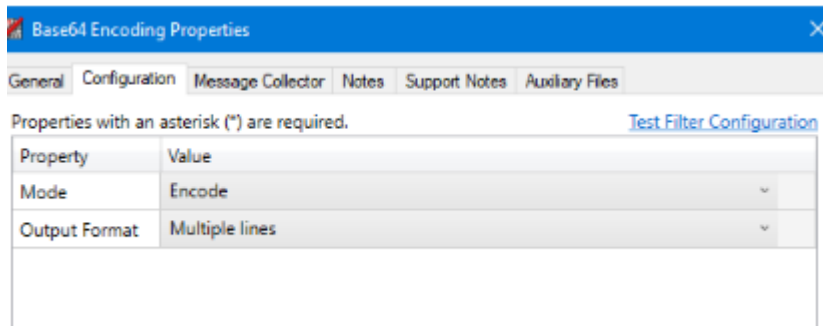
• Base64 Encoding

- The Base64 Encoding filter encodes and decodes between binary data and

Base64 encoded data. Base 64 is a data encoding scheme whereby binary-encoded data is converted to printable ASCII characters. This allows

transmission over protocols designed to deal only with textual data. For example, email as a MIME attachment or within XML.

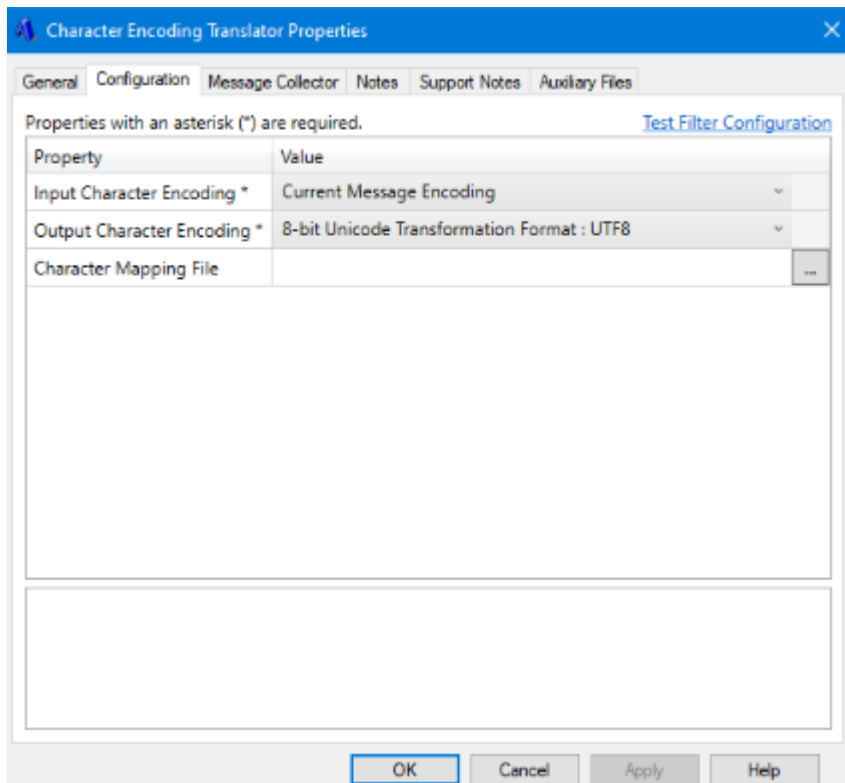
Configuration



- **Mode** - Specifies whether messages should be encoded to or decoded from Base64.
- **Output Format Options** - Specifies whether encoded data should be output as the standard Base64 format (72 characters per line, separated by a carriage return-line feed pair), or as a single line of encoded characters with all carriage return-line feed pairs removed.

- **Character Encoding Translator Filter**

- The Character Encoding Translator filter converts the contents of a message from one character encoding to another. This is useful when messages received from an external system use a particular encoding that must be changed so that extended characters (such as "á") are processed correctly when inserted in a database or sent to a downstream external system.
- The translation consists of reading the message using the specified **Input Character Encoding** property, then writing the output message using the specified **Output Character Encoding** property. To address the problem of maintaining readability in converted messages, you can specify a custom mapping that is applied before the translation, so that alternative characters can be used in cases where there is no match for the input character. The new character must belong to the supported output character set.



- **Input Character Encoding** - the character set used to format the input message. The default is Current Message Encoding.
- **Output Character Encoding** - the character set used to format the output message. The default is 8-bit Unicode Transformation Format : UTF 8.
- **Character Mapping File** - an auxiliary mapping file that will override the global mapping in characterMapping.xml. This is an editable file that provides replacements for characters not covered by the destination encoding.

Content Population Filter

The Content Population filter is used to:

- replace the body of the message with content from a specified file.
- replace the values of specified fields with static (literal) values or message property values.
- perform both of the above actions.

If the **Body Template File** property is supplied, the message body is replaced with the content in the specified file. This option is used to replace the incoming file with a second file, for example, an incoming HL7 message with its matching acknowledgement.

If a message definition that matches the type and structure of an incoming file is specified, the content of specific fields in the message (listed in the **Field**

Mappings field) can be updated with specified values or values retrieved from a message property.

If both a body template file and a definition/field mapping are specified, the incoming message will be replaced by the specified file, after which any specified fields will be updated with the content specified by the field mappings.

Configuration

The screenshot shows the 'Content Population Properties' dialog box with the 'Configuration' tab selected. The dialog has tabs for 'General', 'Configuration', 'Message Collector', 'Notes', 'Support Notes', and 'Auxiliary Files'. Below the tabs, there is a note: 'Properties with an asterisk (*) are required.' and a link 'Test Filter Configuration'. A table lists the properties:

Property	Value
Maximum Concurrency	0
Definition	Academy-HL7v2.4-ADT.s3d (Same as server)
Body Template File	ACK Template.hl7 (New)
Field Mappings *	[[MSH.SendingFacility.NamespaceID,\$(FacilityName)]]

- **Maximum Concurrency** - the maximum number of instances of this filter that can be created and used in parallel. This property can be left at 0 (zero, which means an unlimited number of instances) for this filter, but overall system performance may be improved by setting a limit for the filter.
- **Definition** - the message definition used to identify specific fields whose values are to be updated.
- **Body Template File** - a file whose content replaces the incoming message body.
- **Field Mappings** - only available if a definition file has been specified. The fields to be updated by the values or message properties specified in the value column.

Date Validation Filter

The Date Validation filter is used to compare dates in a message and confirm that they meet certain conditions. For example, it may be used to validate that the admission date in an ADT discharge message occurs prior to the discharge date, or that the date of birth is less than or equal to the current system date and time.

The filter can perform the following comparisons:

- two dates contained in the same message.
- a date contained in a message against a static date; for example, 01/01/2010.
- a date contained in a message against a predefined date; for example, TODAY.

All messages entering this filter must match the definition associated with the filter and must be syntactically valid. Filtering invalid messages can be achieved separately through the use of the EDI Message Validation filter and conditional connectors.

If a message is found to contain one or more dates which are not valid for the specified validation rule, a message property is set containing these validation errors. This property can then be used by the HL7 Acknowledgement Generation filter to generate a negative acknowledgement. The original message will exit the EDI Message Validation filter, regardless of the validity or otherwise of the codes which it contains.

Configuration

The screenshot shows the 'Date Validation Properties' dialog box with the 'Configuration' tab selected. The dialog has tabs for 'General', 'Configuration', 'Message Collector', 'Notes', 'Support Notes', and 'Auxiliary Files'. Below the tabs, there is a note: 'Properties with an asterisk (*) are required.' and a link 'Test Filter Configuration'. A table lists the properties and their values:

Property	Value
Maximum Concurrency	0
Definition	Academy-HL7v2.4-ADT.s3d (Same as server)
Dates To Validate	[[PV1.AdmitDateTime.Time., <, PV1.DischargeDateTime%["%]]
Normalise Time Zones	☒
Offset Time Zones	
Always Apply Time Zone Offset	☐
Fail Action	Send to Error Node
Output Property	DateValidationErrors

Below the table is a section labeled 'Array of dates to validate' with an empty text area.

- **Dates To Validate** - a list of the date pairs to check. See below for additional detail
- **Normalise Time Zones** - whether or not to consider time zones when comparing dates. If this property is set to True, the following properties become available:
 - Offset Time Zones - specify a time zone to be used if the dates to be compared do not have this information.
 - Always Apply Time Zone Offset - If True, the configuration is able to correct a date that is off by a known amount.
- **Fail Action** - the action to take when a message fails date validation. Options: Continue Processing and Send to Error Node.

- **Output Property** - the name of the message property that will be populated with the error list when a message fails date validation. Following processing, this property can be viewed in the Rhapsody Monitoring Console.
- **Dates To Validate**
- Select **Browse** for the **Dates to Validate** property to specify the dates and formats for validation. **Date1** on the left is compared to **Date2** on the right. The date **Format1** and **Format2** can be specified to check the date's format as well as its value; if this is left blank the default HL7 DTTM date format is used. The **Comparator** specifies how the two dates are to be compared while the **Precision/Difference** controls the accuracy of the comparison.

Date1	Format1	Comparator	Date2	Format2	Precision/Difference
PV1.AdmitDateTime.Time		<	PV1.DischargeDateTime[*].Time		h
MSH.DateTimeOfMessage.Time	yyyyMMddHHmmss	<=	TODAY		d

In the above example...

- the patient's hospital admission and discharge dates are compared to an accuracy of hours. If the admit date is earlier than the discharge date, the message will pass validation.
- the message's creation date is checked against today's date to an accuracy of days. If the message's date is the same as or earlier than today's date, and if the date format is 'yyyyMMddHHmmss', the message will pass validation.

Predefined Dates

The following strings can be used to specify a date for comparison:

- TODAY - the current date; time information is ignored
- NOW - the current date and time
- START_OF_MONTH - the first date of the current month
- END_OF_MONTH - the last day of the current month
- START_OF_YEAR - the first day of the current year

- **END_OF_YEAR** - the last day of the current year

If a pre-defined date is specified, any specified date format will be ignored.

Date Format Codes

The following characters can be used in a date/time format string to provide the pattern against which dates are validated. They can also be used to identify the Precision/Difference with which the comparison is made.

- **y** - year (2023)
- **M** - month in year (July & 07)
- **d** - day in month (12)
- **h** - hour in am/pm (0 - 12)
- **H** - hour in day (0 - 23)
- **m** - minute in hour (45)
- **s** - second in minute (32)
- **S** - millisecond (0 - 999)

Error and Hold Queue Filters

Error Queue Filter

The Error Queue filter sends all arriving messages to the Error Queue. These can be viewed, and further processed, by viewing the error queue using the Rhapsody Management Console. It has one configurable property:

- **Error Message** - the explanatory message attached to messages sent to the Error Queue by this filter. Its purpose is to help separate messages sent to the queue from this filter from messages sent to the queue by other methods

The Error Queue filter is often used in conjunction with the Route Message Redirector (discussed in the Communicating with Rhapsody module) or on a route where we want to selectively send additional messages to the error queue for further analysis.

Hold Queue Filter

The Hold Queue filter sends all arriving messages to the Hold Queue in the Rhapsody Management Console. Messages can be deleted from this queue, or viewed and/or edited before being released into the route to continue processing.

It is not recommended that this filter be used in a production environment. It is most useful when debugging a route, as it allows a message's content to be viewed at its different processing stages. This filter has no configurable properties, but must have an output pathway for messages that have been released in the Management Console.

Execute Process Filter

The Execute Process filter executes an external application or script to perform a task or transformation not natively supported by Rhapsody. For example, it might be used to run the GhostScript command to convert a file from PDF to TIFF format prior to sending to a FAX server.

The filter supports two modes of operation:

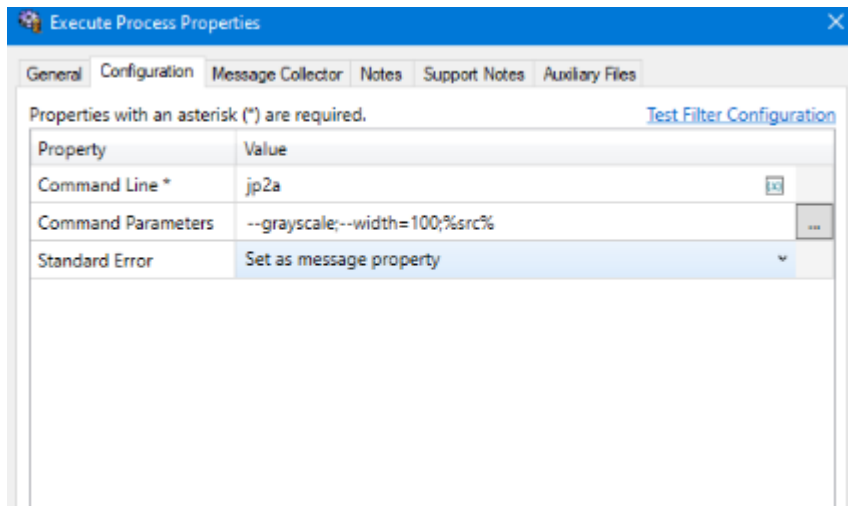
- External process reads from standard input and writes to standard output. In this mode, the process is assumed to operate as with other filters - reading the input message and writing the output message.
- External process requires input from a file, then writes to a file. In this mode, the input file is referenced as %src% and will be written to a temporary file with its file name is substituted in the command. The output file is referenced as %dst% with the filter reading from that file to create the output message.

The command to be run should run from the command line without needing user input to complete the task required or use a GUI (for example, GhostScript or ImageMagick). This may require that the full path to the executable be specified (remember to enclose the path in double quotes (") if it contains spaces).

Where a command reports success or failure information via either the process exit code or the standard error, these should be checked to confirm the message was processed successfully.

Caution is required when using an Execute Process filter to ensure that the process completes and returns the message to the engine in a reasonable time frame. If the process takes a long time to return, it will have a negative impact on engine performance.

Configuration



- **Command Line** - defines the process to be executed. Filters created with older versions of this filter will also include all command parameters at the end of the command line.
- **Command Parameters** - defines the list of parameters to be passed to the command. These could include the %src% and %dst% file references and any other parameters required by the command to run.
- **Standard Error** - identifies what should be done with the standard error output generated by the process. The options are Discard (default), Log to error log and Set as message property. The most appropriate action will depend on the behavior of the process being run.

Published Properties

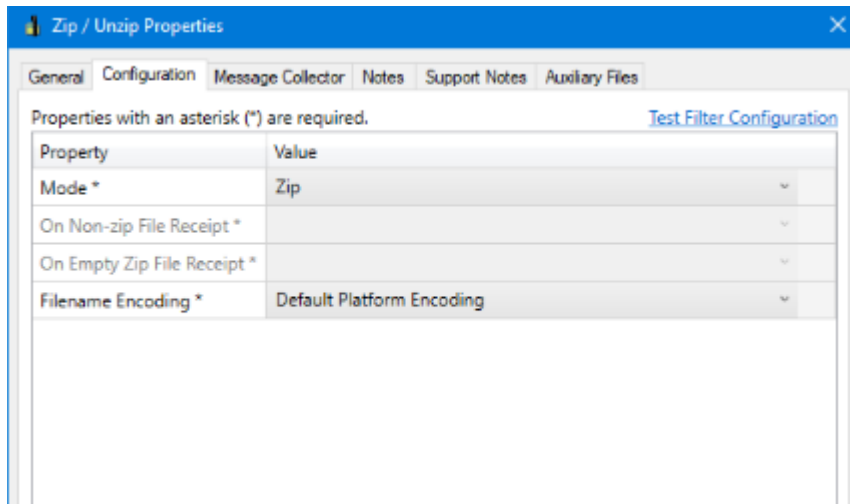
Published properties allow for programmatic error handling if the specified command failed.

- **ExitCode** - the exit code of the spawned process.
- **StandardError** - the error output of the spawned process. This will only be set if the Standard Error property is set to send an error stream to a message property.

• Zip / Unzip Filter

- The Zip / Unzip filter is used to zip separate messages into a single zip file, or to extract individual messages from a zip file. This filter could be used to group messages to be archived into a single ZIP file.
- When this filter is operating in ZIP mode, a message collector (discussed in *Putting it all together*) is used to determine which files belong together in the

ZIP file. In Unzip mode, each file found in the ZIP file will become a message on the Rhapsody route.



- **Mode** - sets whether the action of this filter is to combine messages into a single archive or to separate messages from an archive to individual messages.
- **On Non-zip File Receipt** - (only available in Unzip mode). Sets the action to take if Rhapsody is unable to recognize the incoming file's format, for example if a different compression algorithm has been used. Options are Send to error connector and Send original message to route.
- **On Empty Zip File Receipt** - (only available in Unzip mode). Sets the action to take if an empty archive is received for unzipping. Options are Send to error connector and Send no messages.
- **Filename Encoding** - the character encoding to use for filenames when reading from or writing to a Zip archive. This setting only applies to files within the archive, not the archive itself.
 - In Unzip mode, this setting allows archives containing files, with foreign characters in the filenames, to be processed correctly, provided the input encoding is known.
 - In Zip mode, this setting enables filenames with foreign characters to be archived without the filenames becoming garbled. The default value for this property, regardless of the Mode setting, is Default Platform Encoding.

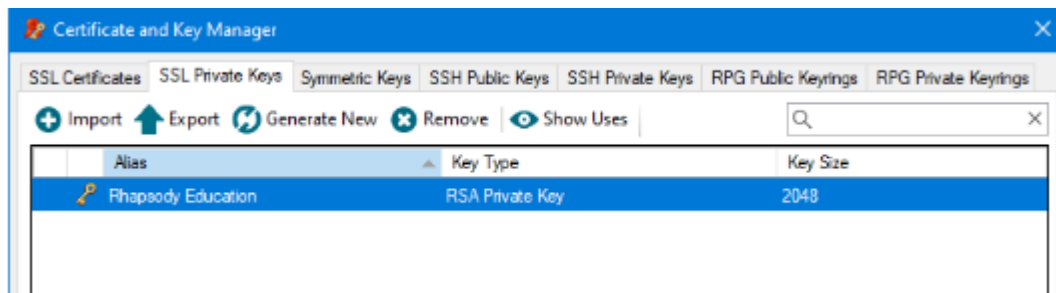
Published Properties

Published properties for the Zip/Unzip filter are:

- **NumberOfBatchedMessages** - indicates how many messages are included in the zip batched message.
- **ZipIndex** - zero-based index of this message within the zip file. This property is populated when *Unzip* has been selected.
- **ZipFullFilename** - filename of this message within the zip file. This property is populated when *Unzip* has been selected.
- **ZipBaseFilename** - base filename without the extension of this message. This property is populated when *Unzip* is selected.
- **ZipSuffix** - file extension of this message. This property is populated when *Unzip* is selected.

Worked Examples

To import the worked examples for this set of filters, first import the **Rhapsody Education.pfx** SSL Private Key from the **Certificate and Key Manager**. The password required for import of the key is "academy".



Each example is built on a separate route and the relevant details are noted in the Route Notes.

The input for each route is proved by a Timer communication point to simplify input.

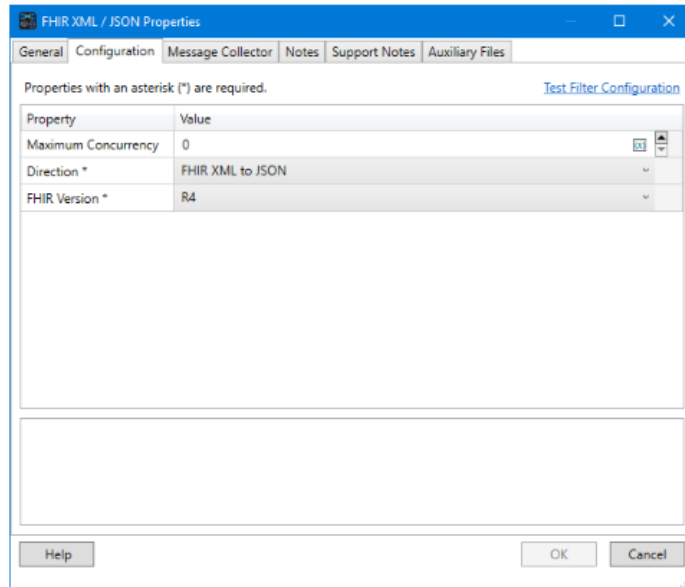
All components are configured for manual start and will require starting to test the configuration.

HL7 Filters

FHIR XML / JSON

HL7 External Code Validation

The FHIR XML / JSON filter converts messages between the [FHIR XML format\(opens in a new tab\)](#) and the [FHIR JSON format\(opens in a new tab\)](#) using the HAPI-FHIR library.



FHIR XML / JSON Properties

The filter supports the following versions of the FHIR specification:

- DSTU2
- STU3
- R4
- R5 (pre-release)

The HL7 FHIR standard is still under active development. Configurations using this filter may need to be updated as the standard (and other systems using the standard) mature.

HL7 Acknowledgement Generation Filter

The HL7 Acknowledgement Generation filter is used to create HL7 Acknowledgements (ACKs) for all incoming messages. It is normally used in the following situations:

- Creating positive acknowledgements for received messages.
- Validating an incoming message's HL7 syntax and generating positive or negative acknowledgements based on the results.
- Creating acknowledgements by using the properties set on the input message as errors.
- **Configuration Properties**
- The **Configuration** tab for the HL7 Acknowledgement Generation filter's **Properties** dialog is shown below:

Property	Value
Message Definition *	Academy-HL7v2.4-ORU.s3d (Same as server)
Response Message Mapping	[[ORUR01,ACKR01]]
Acknowledgement Mode *	Use Properties and ACK
NACK mode	AR
User Description	\$ACKMessage
On message validation failure *	Return Ack only to route
On successful validation *	Return Ack only to route
Message Errors Property Name	\$ACKErrors
HL7 version	Same as input message
Maintain Field Length *	Ignore field lengths
Fields to Copy	[[MSH.SendingApplication,MSH.ReceivingApplication],[MSH.

Rhapsody Associate Review

The use and configuration of the HL7 Acknowledgement Generation filter was described as part of the Rhapsody Associate course.

HL7 Message Modifier

The HL7 Message Modifier filter provides an efficient means for defining the most commonly used modifications to HL7 messages passing through Rhapsody.

A full description of its functionality and configuration can be found in *Messaging*.

Intelligent Mapper

The Intelligent Mapper filter provides an efficient means for constructing a mapping from a set of example HL7 messages.

A full description of its functionality and configuration can be found in *Messaging*.



RPG Filters

RPG Decrypt and Verify Filter

RPG (Rhapsody Privacy Guard) is Rhapsody's implementation of a subset of the OpenPGP standard. The RPG is utilized with two filters that allow you to either Decrypt and Verify or Encrypt and Sign messages.

Encryption/Decryption is a CPU intensive activity. Thus, decrypting numerous concurrent messages can have an adverse impact on the overall performance of the server. This is especially the case when the message size is large. The default value of maximum concurrency property is set to 2. It is recommended that this number not be set much higher.

RPG Decrypt and Verify Filter

The RPG Decrypt and Verify filter can be used for decrypting and/or verifying messages that have been encrypted and/or signed by the RPG Encrypt and Sign filter or compatible external systems.

The filter does not support decrypting and verifying files encrypted and signed using multiple keys.



Property	Value
Maximum Concurrency *	2
Mode	Decrypt and verify
Decrypt Keyring *	
Verify Keyring *	
Passphrase *	

RPG Decrypt and Verify Properties

Maximum Concurrency – The maximum level of concurrency for the filter. 0 (zero) signifies infinite.

- **Mode** – Specifies what action the filter should perform:

- **Decrypt and verify** (default) - the message is both decrypted and verified. The filter automatically determines whether the message was signed and then encrypted or encrypted and then signed, and handles both the cases.
- **Decrypt only** - used when the message has been only encrypted and not signed. Note that if the input message was both signed and encrypted we cannot choose to only decrypt the message.
- **Verify only** - used when the message has only been signed, but not encrypted. If the message is both signed and encrypted, this mode cannot be used.
- **Passphrase only** - used to decrypt the messages that were encrypted using a passphrase.
- **Decrypt Keyring** - The RPG private keyring to decrypt the message. When this property is set, the filter expects incoming messages to be encrypted using the specified key. Unencrypted messages are sent to the Error Queue. The keyring can also be specified via message property by indicating the alias of the keyring. The filter can automatically identify the relevant key from within the given keyring.
- **Verify Keyring** – The RPG public keyring to verify the signature of the message. When this property is set, the filter expects messages to have been signed using the specified key. Unsigned messages are sent to the Error Queue. The keyring can also be specified via message property by indicating the alias of the keyring. The filter automatically identifies the suitable key from within the given keyring.
- **Passphrase** - The passphrase to use when the user does not have keys. If **Mode** is set to Passphrase only, the user needs to provide the passphrase used to decrypt the message. The filter converts the passphrase string into a symmetric key to decrypt the data.

• **RPG Encrypt and Sign**

- The RPG Encrypt and Sign filter encrypts and/or signs messages. These messages can be decrypted and/or verified by the RPG Decrypt and Verify filter or compatible external systems.
- The RPG Encrypt and Sign filter generates a single output when it performs the signing and encryption operations on a message. The filter does not support

producing two separate outputs when encryption and signing, which certain external systems might require.



Property	Value
Maximum Concurrency *	2
Mode	Sign then encrypt
Sign Key *	
Encrypt Key *	
ASCII Armor *	☒
Compression Algorithm *	ZIP
Symmetric Key Algorithm *	AES with 128-bit
Integrity Check *	☒
Hash Algorithm *	SHA-256
Passphrase *	
Filename	
Append Timestamp	☒
Filename Extension	txt

PRG Encrypt and Sign Properties

- **Maximum Concurrency** – The maximum level of concurrency for the filter. 0 (zero) signifies infinite.
- **Mode** – Specifies what mode the filter should operate in:
 - **Sign then encrypt** (default) - the message is first signed and then encrypted. This mode gives extra protection to the signature by encrypting it. This is the more commonly used approach.
 - **Encrypt then sign** - the message is first encrypted and then signed against the original data instead of the encrypted data. Most external systems, however, cannot decrypt and verify the data which is generated using this mode.
 - **Sign only** - the message is signed, but not encrypted, so that the sender can be verified.
 - **Encrypt only** - the message is encrypted, but not signed.

- **Passphrase only** - encryption is performed using a passphrase. The receiver needs to know the passphrase in order to decrypt the data. This mode should be used if the receiver does not have keys.
- The RPG Encrypt and Sign filter does not compress data when performing the Encrypt then sign operation on messages because compression is not efficient on encrypted data. Messages are compressed in all other modes.
- **Sign key** – The RPG private key to sign the message. The key can also be specified using a message property. The value of the message property should be the Alias of the keyring (as shown in Certificate and Key Manager) followed by a key id. Format: alias/keyid.
- **Encrypt Key** – The RPG public key to encrypt the message. The key can also be specified using a message property. The value of the message property should be the Alias of the keyring (as shown in Certificate and Key Manager) followed by a key id. Format: alias/keyid.
- **ASCII Armor** – Whether to use the ASCII armored data format, which is used for receiving systems that only allow encrypted data with ASCII printable characters. Some systems such as Kleopatra are only able to decrypt files that are passphrase encrypted without ASCII Armor enabled.
- **Compression Algorithm** – The algorithm to use to compress the message: ZIP (default), ZLIB, BZIP2.
- **Symmetric Key Algorithm** – The algorithm to use to generate the random session key. The session key will be used to encrypt the message.
- **Integrity Check** – Whether to provide an integrity check by adding a checksum to the encrypted message. The checksum can be used to verify whether the data has been tampered with after it was encrypted.
- **Hash Algorithm** – The hash algorithm to use to sign the message.
- **Passphrase** – If Mode is set to Passphrase only, the passphrase used to encrypt the message must be provided. The filter converts the passphrase string into a symmetric key to encrypt the message.
- **Filename** – The filename that the receiver will see after converting an RPG message into a plain text file. If this property is empty, the current timestamp will be used as the filename. The filename can also be specified via a message property.
- **Append Timestamp** – Specifies whether to append the current timestamp to the filename.

- **Filename Extension** – Specifies filename extension to use for the file (.txt by default). It can be also be specified via a message property.

XML Filters

XML Validation Filter

The XML Validation filter is used to validate XML documents against an XML Schema Definition Language (XSD) schema. This is useful when XML messages are being received from an external system and there is a need to confirm the messages conform to a specification before the message is processed or sent to another system.

The filter returns the original XML message if the message is valid. It includes options to route to the error connector, to continue processing the message, or to return an XML document with full error information if the message is invalid. There are three levels of validation error (*Warn, Error and Fatal Error*) which you can set as the minimum validation failure level. During message validation the filter counts the number of errors which have occurred, in total and for each level of error, and stores these values in the message properties.

The XML Schema Definition Language (XSD) is an XML language for describing and constraining the content of XML documents.

Configuration

The screenshot shows the 'XML Validation Properties' dialog box with the 'Configuration' tab selected. The dialog has a title bar with a close button. Below the title bar are tabs: 'General', 'Configuration', 'Message Collector', 'Notes', 'Support Notes', and 'Auxiliary Files'. The 'Configuration' tab is active. Below the tabs, there is a text label: 'Properties with an asterisk (*) are required.' and a link: 'Test Filter Configuration'. A table with two columns, 'Property' and 'Value', contains the following rows:

Property	Value
Maximum Concurrency	0
W3C XML Schema *	[Dropdown menu]
Minimum Failure Level *	Fatal Error
Continue Checking On Error	☐
On Validation Failure *	Send to error connector
Validation Options	[Dropdown menu]

Below the table are two empty text areas. At the bottom of the dialog are four buttons: 'OK', 'Cancel', 'Apply', and 'Help'.

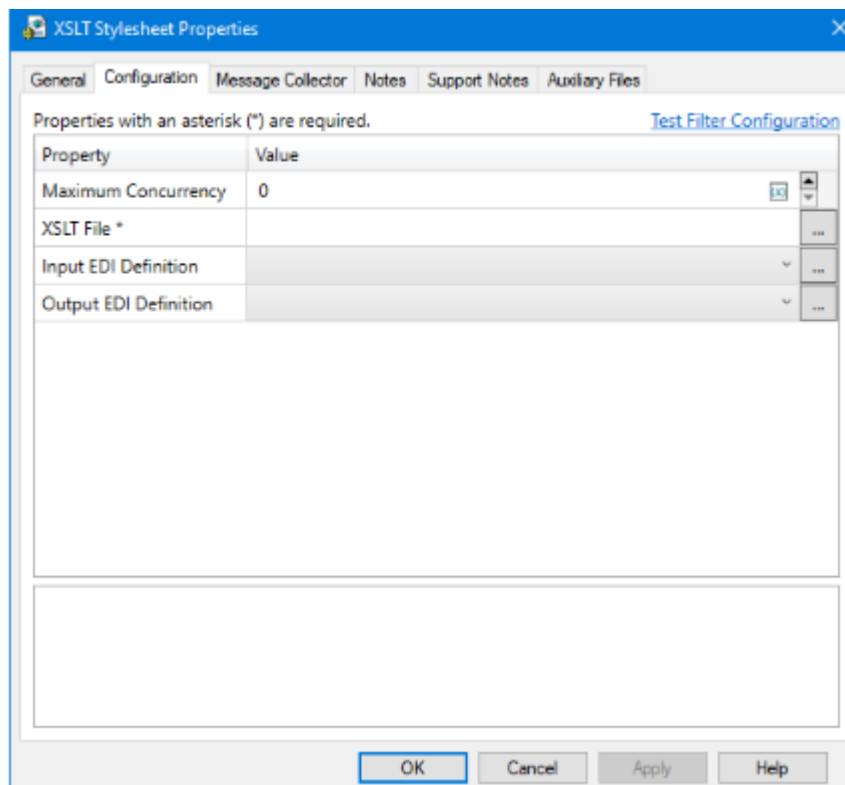
- **W3C XML Schema** - The W3C XML Schema (.xsd) to be used to validate the input XML message.
- **Minimum Failure Level** - Options: Warn, Error, Fatal Error. Default: Fatal Error. The minimum level that the filter decides the message is invalid.
- **Continue Checking On Error** - Options: True or False. Default: False
 - If True, the filter continues validation checking to the end of the message, even if a minimum failure level error occurs. It then decides the message validation failure once checking is completed.
 - If False, the filter stops checking as soon as a minimum failure level error occurs.
- **On Validation Failure** - Options: Send to error connector, Continue processing on route, Send error report to route. Default: Send to error connector
The action to be performed if validation fails.
 - If Send to error connector, the message is sent to the error connector.
 - If Continue processing on route, the message is routed to the next route component to continue processing.
 - If Send error report to route, the body of the message is replaced by an XML document containing full error information along with the original message. This message is routed to the next route component.
- **Validation Options** - XSD validation checking options. All options set to False by default. If a particular checking option needs to be turned on, select **Browse**, select the option and set its value to True. The checking options are:
 - Full schema constraint checking - enable full schema grammar constraint checking, including any checking which may be time-consuming or memory intensive. Particle unique attribution constraint checking and particle derivation restriction checking are controlled by this option. This option checks the schema grammar itself for additional errors that are time consuming or memory intensive. It does not affect the level of checking performed on document instances that use schema grammars.
 - Warn on re-declared duplicate attribute - report a warning for duplicate attribute declaration.
 - Warn on re-declared duplicate entity - report a warning for duplicate entity declaration.
 - Allow Java encoding names in XMLDecl and TextDecl line - if True, the encoding of the file may be specified as a Java encoding name as well as

the standard ISO encoding name. Be aware that other XML parsers may not be able to use Java encoding names. If False, an error will be generated if Java encoding names are used.

XSLT Stylesheet Filter

The XSLT Stylesheet filter applies an XSLT filter to an XML document. This allows the XML document to be transformed, with the output document formatted as plain text, XML, or HTML. Rhapsody supports version 1.0 of the XSLT specification.

Configuration



XSLT File - the XSLT file that will perform the transformation on each message that passes through the filter (there are numerous tutorials available online that provide an introduction to XSLT).

XML-FO Filters

XSL-Formatting Objects (XSL-FO) is a language that describes how an XML file should be converted into one of a number of different paginated document formats, such as PDF or PostScript.

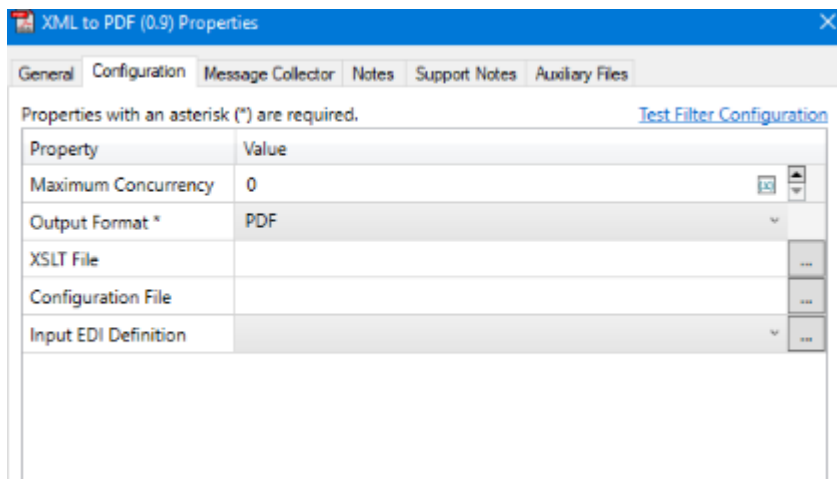
The following filters all use the XSL-FO language to transform an XML message into either a PDF or RTF document. Rhapsody uses the [Apache-FOP](#)(opens in a new tab) library to perform the translation. The version of the Apache-FOP used depends on filter being used.

XML to PDF

The XML to PDF (0.2) and XML to PDF (0.9) filters convert an XML document into either a PDF, PS, PCL, or text document. The XML to PDF (0.2) filter uses version 0.2 of Apache Formatting Objects Processor (FOP), while the XML to PDF (0.9) filter uses version 0.9.

Configuration

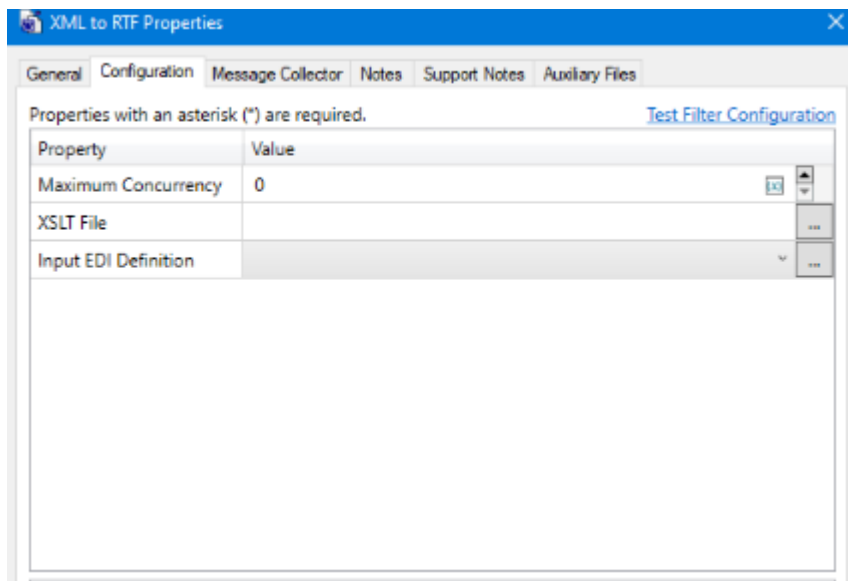
The following properties are common to both versions of this filter.



- **Output Format** - options: PDF, Postscript, PCL, Text. Default: PDF

This property sets the format of the output message. Note that Postscript is required if using this filter in conjunction with the **Printer** communication point.

- **XSLT File** - the XSLT file used to transform the input message from an XML data document to a PDF file. This property is not required if the incoming message is already in XSL:fo format.
- **Configuration File** - an optional XML FOP configuration file used to add extra fonts to the FOP XSL:fo processor.
- **Input EDI Definition** - the EDI message definition for the input file. If specified, the filter converts the incoming file to XML then PDF format without requiring further user action.
- **XML to RTF**
 - The XML to RTF filter converts an XML document into RTF (Rich Text Format).



- **XSLT File** - the XSLT file used to transform the input message from an XML data document to an RTF file. This property is not required if the incoming message is already in XSL:fo format.
- **Input EDI Definition** - the EDI message definition for the input file. If specified, the filter converts the incoming file to XML then RTF format without requiring further user action.

• Web Services Signing

- The Web Services Signing filter uses the web services extensions framework to create digital signatures of parts of a XML message (typically this will be a SOAP message). A signed message allows a recipient to confirm that the signed contents of a message have not been tampered with or corrupted and that the message originates from a trusted source.
- The filter receives an XML message, and then uses a private key from the certificate manager to digitally sign parts of the XML document. In addition, the filter can add a timestamp and expiry time (which can also be signed), and allows the certificate used to verify the signature to be included or referenced in a number of ways. The final output of the filter is an XML message with additional elements added containing the signed elements and the certificate.

- Further information on XML signatures can be found here: [http://msdn.microsoft.com/en-us/library/ms996502.aspx\(opens in a new tab\)](http://msdn.microsoft.com/en-us/library/ms996502.aspx(opens in a new tab)). Note that signing relies on namespaced elements and strictly enforces name-spacing in the output. Attention to name-spacing is required when defining the **Elements to Sign** parameters.

Web Services Signing Properties

General Configuration Message Collector Notes Support Notes Auxiliary Files

Properties with an asterisk (*) are required. [Test Filter Configuration](#)

Property	Value
Signing Key *	
Signing Certificate *	
Timestamp *	Generate Timestamp
Expiry Time	600
Signing Algorithm *	SHA256
Digest Algorithm *	SHA256
Elements to Sign	[[Body,http://schemas.xmlsoap.org/soap/envelope/,Content]]
Certificate Reference *	Include Full Certificate

OK Cancel Apply Help

- **Signing Key** - the private key used by the filter to sign the message. It should match the public key in the supplied certificate so that the remote system can successfully validate the signature.
- **Signing Certificate** - the certificate used to verify the signature. The verification itself is not done by the filter, but the configured certificate is included inside the generated XML document in accordance with the **Certificate Reference** property below.
- **Timestamp** - Options: Generate Timestamp, No timestamp. Default: Generate Timestamp.
This property determines whether or not the filter generates a timestamp element as part of the XML document. If no timestamp is selected, then it is still legal for a timestamp to be signed if it was already present.

The default expiry time is 600 seconds (10 minutes), and expiry times of less than 30 seconds are treated as 30 seconds.

- **Expiry Time** - the expiry time, in seconds, for the signed message. The server will reject messages that are too old. This property is required if generating a timestamp.
- **Elements to Sign** - the list of elements to sign. They are found by their qualified name of the specified name and the namespace. A URI is used for the namespace, meaning that elements can be found regardless of the particular namespace prefix being used. The signing type determines whether just the contents of the element are signed (Content), that is just between the tags, or if the entire element is signed (Element).
- **Certificate Reference** - Options: Include Full Certificate, Include Certificate Identifier Only, Include Encrypted Certificate. Default: Include Full Certificate. Specify how the certificate that should be used to verify the message is included in the output XML document.
 - Include Full Certificate - by default the full certificate is included in a BinarySecurityToken tag, which is more or less the certificate in a base-64 encoded form.
 - Include Certificate Identifier Only - includes just the certificate identifier inside a SecurityTokenReference with the issuer name and serial number of the certificate.
 - Include Encrypted Certificate - the encrypted certificate option includes the full certificate in a KeyIdentifier tag under a securitytokenreference tag, but the certificate is first encrypted using the specified private key, and then base-64 encoded.
The private key used by the filter to sign the message. It should match the public key in the supplied certificate so that the remote system can successfully validate the signature.

• **Worked Examples**

- The attached RLC **Examples - Applying Business Logic - XML.rlc** contains worked examples for each of these filters.

Prior to importing this configuration, make sure the SSL Private Key imported for the **Utility Filters** exists.

Each example is built on a separate route and the relevant details are noted in the Route Notes.

The input for each route is proved by a Timer communication point to simplify input.

All components are configured for manual start and will require starting to test the configuration.